

The code, explained

A guided tour of every important function

Companion to MASTER_EXPLAINER.pdf — the key code embedded and explained,
in the order the pipeline runs.

Team: Keyaan · Anup · Lamiya · Asif | 2026-09-28

The code, explained

A guided tour of every important function

Companion to MASTER_EXPLAINER.pdf. For each module we embed the key code (frozen, so it survives later edits) and explain what each part does and why, in the order the pipeline runs. Read MASTER_EXPLAINER first for the concepts; this shows how they're implemented. Team: Keyaan, Anup, Lamiya, Asif.

How to use in a defense: for any file, you should be able to (a) say its one-line job, (b) point at the key function, and (c) give the "one-sentence defense" at the end of each section.

1. src/data.py — load, clean, and keep every photo aligned to its label

The dataset ships with the JPEG bytes embedded. We expose a lightweight metadata table and fetch images on demand. The critical idea is `_row`: a stable index that ties each metadata row to its image through every later shuffle and split.

```
def clean_metadata(df, seed=42):
    out = df.copy()
    out = out.drop(columns=[c for c in DEAD_COLUMNS if c in out.columns])
    before = len(out)
    out = out.drop_duplicates(subset="image_id", keep="first") # remove 123 exact dupes
    n_dupes = before - len(out)
    out = out.sample(frac=1.0, random_state=seed).reset_index(drop=True) # deterministic shuffle
    out.attrs["n_duplicates_removed"] = n_dupes
    return out
```

What it does: drops dead columns, removes duplicate `image_id` rows (so one photo can't be on both sides of a split), and shuffles deterministically (the rows arrive ordered by pollution bin, so a naive head/tail split would be catastrophically biased). `_row` (added in `load_pooled`) is preserved through all of this, so `get_image(ds, row)` always fetches the picture that matches a metadata row.

One-sentence defense: "We dedupe and shuffle once, deterministically, before any split, and every row carries a `_row` index so predictions, labels, and images can never get misaligned."

2. src/audit.py — prove the data is trustworthy before modelling

Two checks matter: perceptual-hash near-duplicate grouping (leakage risk), and the **physics falsification test** — the elegant one that proves images and labels are correctly paired.

```
# transmission should correlate NEGATIVELY with AQI (hazier photo -> higher pollution)
r, p = pearsonr(mean_transmission_per_image, aqi_labels) # measured r ~= -0.14, p ~= 5e-6
```

What it does: if images and labels were mispaired, "how hazy is this photo" would be *unrelated* to the AQI label. Physics predicts a negative relationship; we measured $r \approx -0.14$ (significant) — weak but real and in the right direction, confirming the pairing and foreshadowing why honest R^2 is modest.

One-sentence defense: "Before modelling we falsification-tested the data: haziness correlates negatively with AQI exactly as physics predicts, so the pairing is real."

3. src/splits.py — leakage-safe splits (the honest protocol)

`_group_split` assigns *whole stations* to one split, so no station straddles the boundary.

```
def _group_split(groups, fractions, seed):
    sizes = groups.value_counts()
    rng = np.random.default_rng(seed)
    order = rng.permutation(sizes.index.to_numpy())
    targets = {s: f * sizes.sum() for s, f in zip(SPLIT_NAMES, fractions)}
    current = {s: 0 for s in SPLIT_NAMES}
    assign = {}
    for g in order: # deficit-greedy: send each station to the split most below target
        s = max(SPLIT_NAMES, key=lambda k: targets[k] - current[k])
        assign[g] = s
        current[s] += int(sizes[g])
    return groups.map(assign).to_numpy()
```

What it does: shuffles the stations, then greedily sends each whole station to whichever split is furthest below its target row-count. Because a station is assigned as a unit, it can never appear in two splits — that's what makes `station_grouped` leakage-safe by construction. `make_splits` also builds the random (leaky control), geographic, temporal, and shipped strategies, and asserts the shipped split is station-disjoint.

One-sentence defense: "Whole stations go to one split, so the model is always tested on genuinely unseen locations — the leaky random split is the control we build to measure inflation."

4. src/physics.py — the Dark Channel Prior and the 5-channel input

```
def dark_channel(rgb, patch=15):
    min_over_channels = rgb.min(axis=2) # darkest colour per pixel
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (patch, patch))
    return cv2.erode(min_over_channels, kernel) # local min-filter (erosion)

def transmission_map(img, patch=15, omega=0.95, t_min=0.05):
    rgb = to_float_rgb(img)
    dark = dark_channel(rgb, patch)
    A = atmospheric_light(rgb, dark) # the haze colour
    t = 1.0 - omega * dark_channel(rgb / A, patch) # low t = dense haze
    return np.clip(t, t_min, 1.0)

def assemble_five_channel(rgb01, t01, s01, imagenet_norm=True):
    phys = np.stack([t01, s01], axis=2)
    rgb = (rgb01 - IMAGENET_MEAN) / IMAGENET_STD # RGB in ImageNet stats
    phys = (phys - PHYS_MEAN) / PHYS_STD # physics maps to ~unit scale
    return np.transpose(np.concatenate([rgb, phys], axis=2), (2, 0, 1)) # (5, H, W)
```

What it does: the dark channel is the darkest colour in each small patch; haze lifts it, so $1 - \omega \cdot \text{darkchannel}$ estimates transmission (LOW where haze is dense). `assemble_five_channel` stacks normalised RGB with the transmission and inverted-saturation maps into the (5, H, W) tensor the network reads. The maps are cached to disk (`build_map_cache`) so the expensive Dark Channel Prior runs once, not every epoch.

One-sentence defense: "We hand the network two physics maps — transmission and inverted saturation — so it reads the haze signal directly instead of rediscovering atmospheric optics from pixels."

5. src/model.py — EfficientNet-B0 + non-crossing quantiles + a mean-seeking point head

```
class MonotoneQuantiles(nn.Module):
    def forward(self, feats):
        z = self.fc(feats) # (B, n)
        base = z[:, :1] # lowest quantile, unconstrained
        increments = F.softplus(z[:, 1:]) # all strictly > 0
        steps = torch.cumsum(increments, dim=1) # running total
        return torch.cat([base, base + steps], dim=1) # ascending BY CONSTRUCTION
```

What it does: the low/median/high outputs can never cross, because each is the previous plus a strictly-positive `softplus` increment. No penalty term, no post-hoc sorting. The `PM25QuantileNet` also has a **point head** (`nn.Linear`) trained to predict a *standardised* target; buffers `y_mean/y_std` store the train-set stats so we can de-standardise back to AQI at inference. The backbone is built with `num_classes=0` so it returns pooled features (which the abstention OOD gate also uses).

One-sentence defense: "The three quantile heads are ordered by construction via cumulative `softplus`, and a separate MSE point head estimates the mean (which R^2 rewards) rather than the median."

6. src/losses.py — pinball loss + the mean-seeking point loss

```
def pinball_loss(preds, target, quantiles):
    q = torch.as_tensor(quantiles).unsqueeze(0) # (1, Q)
    error = target - preds
    return torch.maximum(q * error, (q - 1.0) * error).mean() # asymmetric penalty

def combined_loss(out, y_raw, quantiles, point_weight, huber_delta, y_mean, y_std, point_loss="mse"):
    y_log = torch.log(y_raw.clamp_min(1e-6))
    loss = pinball_loss(out["quantiles"], y_log, quantiles) # quantiles in LOG space
    if "point" in out and point_weight > 0:
        pt_target = (y_raw - y_mean) / y_std # standardised target
        point_term = F.mse_loss(out["point"], pt_target) # MSE -> estimates the MEAN
        loss = loss + point_weight * point_term
    return loss
```

What it does: for $\tau=0.95$ the pinball loss penalises under-prediction $19\times$ more than over-prediction, so that head learns the top of the range; $\tau=0.05$ mirrors it; $\tau=0.50$ is symmetric. The quantiles train on $\log(\text{AQI})$ (stable on a skewed target). The point head uses **MSE on a standardised target** — whose minimiser is the conditional **mean**, exactly what R^2 rewards (using Huber here would learn the median and depress R^2 — that was a real bug we fixed).

One-sentence defense: "Pinball loss teaches each percentile with an asymmetric penalty; MSE on the standardised target makes the point head mean-seeking, which is what R^2 measures."

7. src/dataset.py — 5-channel tensors, log target handling, balanced sampling

```
def balanced_weights(targets_aqi, power=0.5, edges=AQI_BAND_EDGES):
    band = np.digitize(targets_aqi, edges[1:-1])
    counts = np.bincount(band, minlength=len(edges) - 1).astype(float)
```

```
freq = counts[band]
return (1.0 / np.maximum(freq, 1.0)) ** power # oversample rare (high-AQI) bands
```

What it does: `__getitem__` returns the cached 5-channel tensor + the **raw** AQI (the loss applies log/standardise). Geometric-only augmentation (flips/rotations/crops) is applied identically to all 5 channels — **no photometric augmentation**, because changing brightness/contrast would corrupt the exact haze signal we rely on. `balanced_weights` powers a `WeightedRandomSampler` on the **train loader only**, so the extremes are actually learned while calibration/test keep the natural distribution (honest metrics).

One-sentence defense: "Train-only class-balanced sampling shows the model the rare high-AQI photos; calibration and test stay natural so the guarantee and the reported numbers remain honest."

8. `src/train.py` — the loop, checkpointing, and the eval collectors

```
for epoch in range(max_epochs):
    net.train()
    for x, y in loaders["train"]:
        loss = combined_loss(net(x), y, quantiles, point_weight, huber_delta, y_mean, y_std, point_loss)
        scaler.scale(loss).backward(); scaler.step(opt); scaler.update()
        if ema is not None: ema.update_parameters(net)
    cal_mae = point_mae_from_outputs(collect_outputs(eval_model, loaders["cal"], device), y_mean, y_std)
    if cal_mae < best_mae - 1e-4: # EARLY STOP on calibration point-MAE
        best_mae = cal_mae; torch.save({...}, ckpt_path) # save the BEST checkpoint

def collect_features(net, loader, device): # backbone features for the OOD gate
    ...
    feats = net.backbone(x.to(device)); ...
```

What it does: trains with AdamW + cosine warm-up + mixed precision, evaluates the ****calibration point-MAE each epoch, and saves the best checkpoint**** (not the last) — a past bug was evaluating the last epoch. `collect_outputs` returns predictions; `collect_features` returns backbone features (for C3). Optional EMA/TTA/ensemble are wired here and in `leakage.py`.

One-sentence defense: "We select on the calibration set and evaluate the best checkpoint once — no tuning on test."

9. `src/calibrate.py` — the conformal guarantee (CQR)

```
def conformal_Q(preds_cal, y_cal, coverage=0.90):
    lo, hi = preds_cal[:, 0], preds_cal[:, 2]
    E = np.maximum(lo - y_cal, y_cal - hi) # nonconformity: how far outside the interval
    level = min(1.0, np.ceil((len(E) + 1) * coverage) / len(E)) # finite-sample correction
    return float(np.quantile(E, level, method="higher"))

def apply_conformal(preds, Q):
    out = preds.astype(float).copy()
    out[:, 0] = np.maximum(out[:, 0] - Q, 0.0) # widen low (clip at 0)
    out[:, -1] = out[:, -1] + Q # widen high
    return out
```

What it does: `E_i` is how far each calibration truth fell outside its predicted range. `Q` is the finite-sample 90th percentile of those misses. Widening every future interval by `Q` buys back exactly the coverage the raw model was missing — a distribution-free ~90% guarantee for **any** model, under exchangeability.

One-sentence defense: "Q is our model's own typical miss on held-out data; padding every interval by `Q` gives a provable ≥90% coverage guarantee that holds for any model."

10. `src/recalibrate.py` — isotonic de-biasing

```
def fit_isotonic(cal_point, cal_true):
    iso = IsotonicRegression(out_of_bounds="clip", y_min=0.0)
    iso.fit(cal_point, cal_true) # monotone prediction -> truth map, on CALIBRATION only
    return iso
```

What it does: learns a monotone map that corrects the systematic under-prediction of high AQI (e.g. "when it says 260, truth averages 340"). Fit on calibration, applied to test — never fit on test. It removes bias without scrambling the ranking, which directly improves R^2 and MAE.

One-sentence defense: "A monotone calibration-fit map removes the point head's systematic tail bias; it can't reorder predictions, so it's low-risk."

11. `src/metrics.py` — both R^2 definitions, so nobody can quote the looser one

```
def point_metrics(y_true, y_median):
    return {
        "MAE": mean_absolute_error(y_true, y_median),
        "RMSE": np.sqrt(mean_squared_error(y_true, y_median)),
        "R2": r2_score(y_true, y_median), # strict coeff. of determination
```

```

    "r2_pearson": pearsonr(y_true, y_median).statistic ** 2, # squared correlation (looser)
    "Spearman": spearmanr(y_true, y_median).statistic,
}

```

What it does: reports the strict R^2 (`r2_score`, our headline) **and** squared-Pearson side by side, plus MAE/RMSE/Spearman. The strict R^2 is what we defend; printing both pre-empts the "which R^2 ?" trap. The identity $R^2 \approx 1 - (\text{RMSE}/\text{SD})^2$ is our built-in self-check (every results row reconciles).

One-sentence defense: "We headline the strict coefficient of determination and also print squared-Pearson, so the metric is unambiguous."

12. `src/leakage.py` — the headline experiment + the causal proof

```

def contaminated_vs_clean(sp_random, test_df, dup_df, station_col="station_id"):
    train = sp_random[sp_random["split"] == "train"]
    train_stations = set(train[station_col])
    train_groups = set(dup_df.merge(train[["_row"]], on="_row")["dup_group"])
    t = test_df.merge(dup_df[["_row", "dup_group"]], on="_row", how="left")
    contaminated = t[station_col].isin(train_stations) | t["dup_group"].isin(train_groups)
    # score R2/MAE/RMSE/SD_y on the contaminated vs the clean subset
    return {"contaminated": _score(contaminated), "clean": _score(~contaminated), ...}

```

What it does: `train_and_evaluate` trains one split and returns metrics + a per-test-row frame (with an assert that the frame's label equals the loader's target — the alignment guard). `contaminated_vs_clean` partitions the *random* test set into leaked (station or near-duplicate also in train) vs clean, and scores each — giving the causal proof (contaminated 0.762 vs clean -0.86; the -0.86 is variance-amplified, so it also reports `SD_y` and RMSE).

One-sentence defense: "Inside the leaky split we separate leaked from genuinely-unseen photos; the whole advantage is in the leaked ones, which proves the inflation is leakage, not an easier test."

13. `src/crossval.py` — leakage-safe GroupKFold cross-validation

```

def make_cv_folds(df, n_splits=5, cal_frac=0.1875, seed=42, station_col="station_id"):
    gkf = GroupKFold(n_splits=n_splits)
    for k, (train_idx, test_idx) in enumerate(gkf.split(df, groups=df[station_col])):
        # test = this fold's stations; carve a station-grouped cal set from the remaining stations
        ...

```

What it does: GroupKFold by `station_id` guarantees every station is tested exactly once and never appears in its own fold's training data (0 straddling). `summarize_cv` reports mean, SD, and a Student-t CI (with `ci_half` separated from SD), and `nadeau_bengio_ci` gives the rigorous corrected-resampled-t for repeated CV. Result: $R^2 = 0.385 \pm 0.19$, coverage 0.895.

One-sentence defense: "Plain k-fold would re-leak same-station photos; grouping by station keeps every fold honest, turning one number (0.22) into $0.385 \pm \text{CI}$."

14. `src/ceiling.py` + `src/aqi.py` — the C2 error ceiling

```

def error_ceiling(var_epsilon, var_labels, deviations=None, coverage=0.90):
    r2_max_raw = 1.0 - var_epsilon / var_labels # R2_max = 1 - Var(eps)/Var(y)
    floor = empirical_interval_floor(deviations, coverage) # 5-95 percentile within-day spread
    return {"R2_max": float(np.clip(r2_max_raw, 0, 1)), "R2_max_is_upper_bound": True,
            "interval_width_floor": floor, ...}

```

What it does: `within_day_aqi_variance` converts each hourly OpenAQ reading to AQI and takes the within-day variance; `level_reweighted_var_epsilon` reweights that noise curve to our dataset's own pollution mix (with an optional **conservative** high-band imputation); `error_ceiling` forms $R^2_{\text{max}} = 1 - \text{Var}(\epsilon)/\text{Var}(y)$. Measured ≈ 0.98 (0.979–0.982). `pm25_to_aqi` (in `aqi.py`) is the official EPA piecewise-linear breakpoint conversion. The code flags the ceiling **optimistic** when high-AQI coverage is incomplete.

One-sentence defense: "The ceiling is $1 - \text{within-day-noise} / \text{label-variance} \approx 0.98$ — an upper bound showing label noise is only ~2% of the variance, so our gap is the visual task, not the labels."

15. `src/abstain.py` — the two abstention gates

```

def fit_mahalanobis(feats_cal_good):
    lw = LedoitWolf().fit(feats_cal_good) # shrinkage Gaussian on good-image features
    return {"mean": lw.location_, "precision": lw.precision_}

def decide(ood_score, log_width, tau_ood, tau_width):
    if ood_score > tau_ood: # GATE 1: out-of-distribution (checked FIRST)
        return {"answer": False, "reason": "unusable / out-of-distribution photo"}
    if log_width > tau_width: # GATE 2: too uncertain
        return {"answer": False, "reason": "model too uncertain (interval too wide)"}
    return {"answer": True, "reason": "confident, in-distribution"}

```

What it does: the OOD gate scores feature-space distance from the training photos (Mahalanobis), with handcrafted brightness/detail/sky checks as backup; the uncertainty gate uses the **log** interval width. Thresholds come from calibration only (ood_threshold = 95th percentile → 5% false-refusal budget; width_threshold from the risk-coverage curve). risk_coverage_curve/aurc/ood_metrics evaluate it.

One-sentence defense: "We refuse out-of-distribution photos by feature distance first, then uncertain ones by log interval width, with thresholds fixed on calibration at a 5% false-refusal budget."

16. src/inference.py — one photo, end to end (the demo)

```
def predict(image, bundle):
    rgb01, t01, s01 = physics.compute_maps(image)
    x = physics.assemble_five_channel(rgb01, t01, s01)
    out = bundle["net"](torch.from_numpy(x)[None])
    q_aqi = np.exp(out["quantiles"].numpy()) # invert the log target
    low, high = max(q_aqi[0,0] - Q, 0), q_aqi[0,2] + Q # conformal widening
    point = apply_isotonic_knots(point_to_aqi(out["point"], y_mean, y_std), ...) # de-bias
    gate = A.decide(ood_score, log_width, tau_ood, tau_width) # answer / abstain
    return {"aqi": point, "low": low, "high": high, "epa_category": ..., "answer": gate["answer"], ...}
```

What it does: runs the whole trained pipeline on a single photo and returns the AQI, the interval, the EPA category, the answer/abstain decision, and the physics maps. It loads a self-contained **bundle** (checkpoint + Q + stats + isotonic knots + thresholds + Mahalanobis params), so the demo needs no dataset at serve time. app/app.py wraps this in Gradio for the Hugging Face Space.

One-sentence defense: "The demo runs the identical trained pipeline on one photo — physics input, calibrated interval, de-biased point, and the abstention gate — from a self-contained bundle."

How the files map to the pipeline (quick index)

Stage	Files
Load + audit	data.py, audit.py
Leakage-safe split	splits.py
Physics input	physics.py, dataset.py
Model	model.py, losses.py
Train	train.py
Calibrate + de-bias	calibrate.py, recalibrate.py
Measure	metrics.py
Leakage experiment	leakage.py
Cross-validation	crossval.py
C2 ceiling	ceiling.py, aqi.py
C3 abstention	abstain.py, train.collect_features
Demo	inference.py, app/app.py